Improving Dynamic Programming for Knapsack

Date: 2026-11-07

Abstract

We study how to make the dynamic program (DP) for the 0-1 knapsack problem faster. Besides the DP from class, we implement two methods from the book by Kellerer, Pferschy and Pisinger: one keeps only the useful states of each row in a sorted list, and one also removes states with an upper bound. For the bonus part we implement the method of Polak, Rohwedder and Węgrzycki, whose time depends on the largest weight instead of the capacity. All methods give the same answers on 3,332 test cases. Our experiments show that the lists only help when the capacity is very large, that the upper bound makes most inputs much faster but fails on some hard types of data, and that the bonus method is the most stable one when the weights are small.

Contents

1 Introduction **3**

2 Algorithm Specification **3**

2.1 DP-1 . . . 3

2.2 DP-with-Lists . . . 4

2.3 DP-with-Upper-Bounds . . . 4

2.4 Small-Weight Method (Bonus) . . . 5

2.5 Correctness . . . 6

3 Testing Results **6**

3.1 Method . . . 6

3.2 Correctness Tests . . . 7

3.3 Test Data . . . 7

3.4 Timing Method . . . 8

3.5 Experiment 1: Number of Items . . . 8

3.6 Experiment 2: Weight Range . . . 9

3.7 Experiment 3: Many Small Items (Bonus) . . . 10

3.8 Experiment 4: Largest Weight (Bonus) . . . 11

4 Analysis and Comments **12**

4.1 Time Complexity . . . 13

4.2 Space Complexity . . . 13

4.3 Discussion of Test Results . . . 13

4.4 Limitations and Improvements . . . 14

5 Conclusion **15**

6 References **15**

7 Source Code Appendix **15**

1 Introduction

In the 0-1 knapsack problem we are given n items and a knapsack with capacity c. Item i has a positive integer weight w_i and a non-negative integer profit p_i. We choose a set of items whose total weight is at most c and whose total profit is as large as possible. Each item is either packed once or not packed.

Our program reads `n c` on the first line and then one line `w p` for each item. Weights and profits are at most 10^9. It prints only the largest total profit, not the chosen items.

The DP from class fills a table with one row for each item and one column for each capacity from 0 to c, so it always takes $\Theta(nc)$ time. The trouble is that c is a number in the input, not the number of items. For example, 1,000 items with weights up to 10^6 give a capacity of about 2.5×10^8, so the table has 2.5×10^{11} entries. On our machine this takes more than a minute, and one row alone needs 2 GB of memory. However, most of these entries never matter for the answer, so a better method should skip them.

Following Sections 3.4 and 3.5 of the book by Kellerer, Pferschy and Pisinger [1], we first store each row as a short list of useful entries, and then also throw away entries that can not lead to a better answer. For the bonus part we implement the $O(n + w^3)$ method of Polak, Rohwedder and Węgrzycki [2], where w is the largest weight; its time does not depend on c at all. We generate five types of test data, check that all methods agree, and measure how their times change with the number of items, the size of the weights and the type of data.

2 Algorithm Specification

2.1 DP-1

Let $z_j(d)$ be the best profit using only the first j items and capacity d. Then $z_0(d) = 0$, and for $j \geq 1$

$$z_j(d) = \max\{z_{j-1}(d),\ z_{j-1}(d - w_j) + p_j\},$$

where the second choice is only allowed when $d \geq w_j$. The answer is $z_n(c)$. The book [1] calls this method DP-1.

We keep only one row `z[0..c]`. For each item we update `z[d]` for d from c down to w_j. Going down is the key point: when we compute `z[d]`, the entry `z[d - w]` has not been changed yet, so it still holds the value of the previous row. This gives the same answers as the two-loop table from class, with $O(c)$ memory instead of $O(nc)$. We use DP-1 as the baseline in all experiments.

2.2 DP-with-Lists

Most entries of a DP row are not interesting. If $z_j(5) = z_j(6) = z_j(7)$, the entries at 6 and 7 add nothing, because capacity 5 already gives the same profit with less weight. Section 3.4 of [1] therefore stores each row as a list of states (w, p), where a state is a set of the first j items with weight sum w and profit sum p. We say that a state dominates another one if it is not heavier, not less profitable, and better in at least one of the two [1]. A dominated state never leads to an optimal solution, because the same items added to the better state give a solution that is at least as good [1], so we delete it.

We keep every list sorted by weight. Without dominated states, the profits also increase along the list, so the last state always has the largest profit. One step from list L_{j-1} to L_j has two parts:

1. **Add the item.** Add (w_j, p_j) to every state of L_{j-1}. We stop at the first state whose new weight is more than c; since the list is sorted, all later states are too heavy as well.
2. **Merge.** Merge the old list and the new list like in merge sort, always taking the lighter state first. A state is kept only if its profit is larger than the profit of the last kept state. If it has the same weight as the last kept state, it replaces that state.

The answer is the profit of the last state in L_n. For example, take the instance of Fig. 3.7 in [1] with $c = 9$ (the figure prints each state with the profit first). The first item $(2, 6)$ gives $L_1 = ((0, 0), (2, 6))$. The second item $(3, 5)$ gives the new states $(3, 5)$ and $(5, 11)$. When we merge, $(3, 5)$ is dropped because $(2, 6)$ is lighter and more profitable, so $L_2 = ((0, 0), (2, 6), (5, 11))$.

2.3 DP-with-Upper-Bounds

A state of the list method has fixed which of the first j items are packed, just like a node of a search tree in branch and bound. Section 3.5 of [1] uses this to remove more states. We keep z, the profit of the best real solution found so far. For every state we also compute an upper bound $U(w, p)$, a number that no solution growing from this state can exceed. If $U(w, p) \leq z$, the state can not give anything better than z, so we drop it.

As in [1], the upper bound comes from the LP relaxation, which means that the last item may be packed in part. We first sort the items by efficiency p_i/w_i from high to low. Then the best way to fill the remaining room $c - w$ with items $j + 1, \ldots, n$ is simply to take them in order, until the first item that does not fit, and to take a part of that item. Packing a part of an item can only help, so this gives an upper bound. With prefix sums of the weights and profits, we find the last item that fits by binary search. For example, if a state is $(2, 3)$, $c = 10$ and the remaining items are $(4, 8)$, $(4, 6)$, $(5, 5)$ in this order, the room 8 holds the first two items, so $U = 3 + 8 + 6 = 17$. If we already know a solution with profit 18, this state is dropped.

The step for item j has three parts:

1. **Grow.** Build L_j from L_{j-1} exactly as in DP-with-Lists.
2. **Update the best solution.** Every state is a real solution, so we set z to the larger of z and the profit of the last state.
3. **Bound.** Remove every state with $U(w, p) \leq z$.

At the start, z is the profit of the greedy solution, which goes through the sorted items and packs every item that still fits. The loop stops early when the list becomes empty, and the answer is z.

Sorting also changes the order in which the items are processed, and the order alone may change the number of states. So we also run DP-with-Lists on the sorted items, called "lists, sorted" below. This shows how much of the speed-up comes from the bound and how much from the order.

2.4 Small-Weight Method (Bonus)

Let w be the largest weight. The idea of [2] is that a good greedy solution is already almost optimal, so we only need to repair a small part of it, and the size of this part depends on w, not on c. We first drop items that are heavier than c. If all remaining items fit together, the answer is their total profit.

Greedy prefix. We take the items in order of efficiency until the next item does not fit. This set G is called the maximal prefix solution [2], and the room left is less than w. We do not sort all items to find G. Instead we work like quickselect, the selection version of quicksort: we pick a random pivot, and if the items that are more efficient than the pivot already weigh more than the room, G ends among them; otherwise we take all of them and continue with the rest. The expected time is $O(n)$.

Proximity. Polak et al. show that some optimal solution differs from G in at most $2w$ items [2]. The idea is an exchange argument: if the two solutions differed in more items, we could find a group of items removed from G and a group of items added to G with the same total weight. The removed items are at least as efficient as the added ones, so swapping these two groups back gives an optimal solution closer to G. Since each item weighs at most w, the removed items weigh at most $2w^2$ in total. For example, if all weights are at most 3, at most 6 items change, with total weight at most 18. So we only need to try "remove weight exactly k from G, then add weight at most $k + \Delta$ from the other items" for $k = 0, \ldots, 2w^2$, where $\Delta = c - \sum_{i \in G} w_i$ is the room left by G.

Best removal and best addition. These are two small knapsack problems with capacity T of about $2w^2$: the smallest profit lost by removing items of G with total weight exactly k, and the largest profit gained by adding other items with total weight exactly y. We solve each one weight at a time. Among items of the same weight h, using j of them is best with the j most profitable ones, so their value is a prefix sum b_j of the profits sorted from high to low. Each step of b is no

larger than the step before; such a sequence is called concave. Adding all items of weight h to the current array f means computing

$f'(x) = \max_j\{f(x - jh) + b_j\}$

for every x, which is called a max-plus convolution. Done directly, this would take $O(T^2/h)$ time for one weight. However, because b is concave, the best starting point $x - jh$ never moves backward as x grows, and in this case the SMAWK algorithm [3] finds all the maxima in linear time. For values of j outside the range of b we use a very small value that still keeps b concave, so SMAWK can be used without special cases.

Combine. The answer is the profit of G plus the best value of "gain with weight at most $k + \Delta$ minus loss with weight exactly k" over all k. Every such choice weighs at most c, so it is a real solution.

2.5 Correctness

DP-1 is the recurrence above, and the downward loop keeps the old row values that the recurrence needs. DP-with-Lists only deletes dominated states, which never lead to an optimal solution [1]. So for every capacity d, the list L_j still holds a state with weight at most d and profit $z_j(d)$, and the last state of L_n has profit $z_n(c)$.

DP-with-Upper-Bounds only deletes more states. A state is deleted only when its upper bound is at most z, so no real solution that grows from it beats z. Since z is always the profit of a real solution, the final z is optimal.

For the small-weight method, the result of [2] says that one optimal solution has the form "remove weight $k \leq 2w^2$ from G, add weight at most $k + \Delta$" . Our two arrays give the best removal and the best addition for every weight, so our best combination is at least as good as that solution, and it is also a real solution. The arrays are correct because the best j items of one weight are always the j most profitable ones, and SMAWK returns the exact maxima [3].

3 Testing Results

3.1 Method

All tests were run on macOS 26.6 with an Apple M5 (10 cores), Apple Clang 21.0.0 and Python 3.9. We compile with `-O2`, and also with AddressSanitizer and UndefinedBehaviorSanitizer to find memory errors.

We check the C++ program against two independent checks written in Python. For $n \leq 12$ we try all subsets. For larger inputs with a small capacity we use a separate Python version of DP-1. When an input is too large for both, all methods must print the same value. The command `knapsack all` runs every method on the same input, so one test checks all of them.

3.2 Correctness Tests

Test group (count)	Result	Purpose
Hand-made cases (11)	PASS	No items, capacity 0, every item too heavy, all items fit, one item fills the bag, equal efficiencies, zero profits, greedy is not optimal, 300 copies of one item, and the example of Fig. 3.7 in [1].
Random small inputs (3,000)	PASS	$n \leq 12$ and weights up to 100, with capacity from 0 to above the total weight; compared with all subsets. Small ranges give many ties.
Many small items (200)	PASS	20 to 200 items with weights 1 to 8, so each weight of the bonus method has many items; compared with Python DP.
Generated inputs (90)	PASS	All five data types, n from 50 to 200, R from 10 to 1,000; compared with Python DP when $nc \leq 3 \times 10^5$, otherwise all methods must agree.
Large inputs (6)	PASS	Capacity about 2.5×10^9, which DP-1 and the bonus method refuse, so the list methods must agree; and 20,000 items with weights up to 10, where all five methods must agree.
Bad inputs (5)	PASS	Missing items, weight 0, negative capacity, negative profit and empty input must be rejected.
Same data everywhere (20)	PASS	The generator must print exactly what a Python version of the standard random engine gives, so the data does not depend on the compiler.

All 3,332 cases pass. The sanitizer build runs a smaller set of 606 cases and reports no error. The timing script below also checks that all methods print the same value on every input it measures.

3.3 Test Data

Our generator `code/gen.cpp` makes five types of data that are common in the knapsack literature [4]. Here R is the weight range and $U(a, b)$ is a random integer from a to b:

1. **uncorrelated**: $w_i = U(1, R)$ and $p_i = U(1, R)$, chosen independently.
2. **weak** (weakly correlated): $w_i = U(1, R)$ and $p_i = U(w_i - R/10,\ w_i + R/10)$, at least 1.
3. **strong** (strongly correlated): $w_i = U(1, R)$ and $p_i = w_i + R/10$.

4. **inverse** (inverse strongly correlated): $p_i = U(1, R)$ and $w_i = p_i + R/10$.
5. **subset** (subset sum): $w_i = U(1, R)$ and $p_i = w_i$.

The capacity is half of the total weight. In the correlated types the profit of a set depends mostly on its total weight; for example, in the strong type a set with weight W and k items has profit $W + kR/10$. So many different sets have almost the same profit, and it is hard to prove that one of them is the best. The generator takes its random numbers only from `mt19937_64`, whose output is fixed by the C++ standard, and turns them into integers with its own short function, so the same arguments give the same data with any compiler.

3.4 Timing Method

Our performance test program `code/bench.cpp` runs one method on one input once as warmup and then 10 more times, and reports the median. It times only the method, not starting the program or reading the input. Every setting uses three inputs (seeds 1, 2 and 3). The tables give the median of the three medians in milliseconds, and the error bars in the figures go from the fastest to the slowest of the three inputs.

We also count the work of each run: the table entries for DP-1, the list states for the list methods, and the convolution entries for the small-weight method. Work does not depend on the machine. To keep the total time near half an hour, we stop running a method on larger inputs once its median work passes a fixed limit (2×10^9 entries for DP-1, 10^8 states for the lists, 3×10^7 for the other two), which is about half a second on our machine. Such cells are marked "–". DP-1 also refuses inputs with $c > 2 \times 10^8$, because its array would need 1.6 GB.

3.5 Experiment 1: Number of Items

We fix $R = 1000$ and use $n = 250, 500, 1000, 2000, 4000$. The capacity is half the total weight, about $250n$, so it grows with n and the DP table grows like n^2. The small-weight method is not run here, because $w \approx 1000$ makes $w^3 = 10^9$ too large. The table gives the median time in ms for $n = 1000$.

Type	DP-1	Lists	Lists, sorted	Bounds
uncorrelated	63.3	194	254	0.128
weak	63.3	469	485	1.76
strong	63.7	930	271	10.4
inverse	76.6	647	461	1.05
subset	63.8	300	367	934

At $n = 4000$ DP-1 takes 1,040 to 1,260 ms on every type, while DP-with-Upper-Bounds takes 1.1 ms (uncorrelated), 4.8 ms (weak), 45 ms (inverse) and 515 ms (strong).

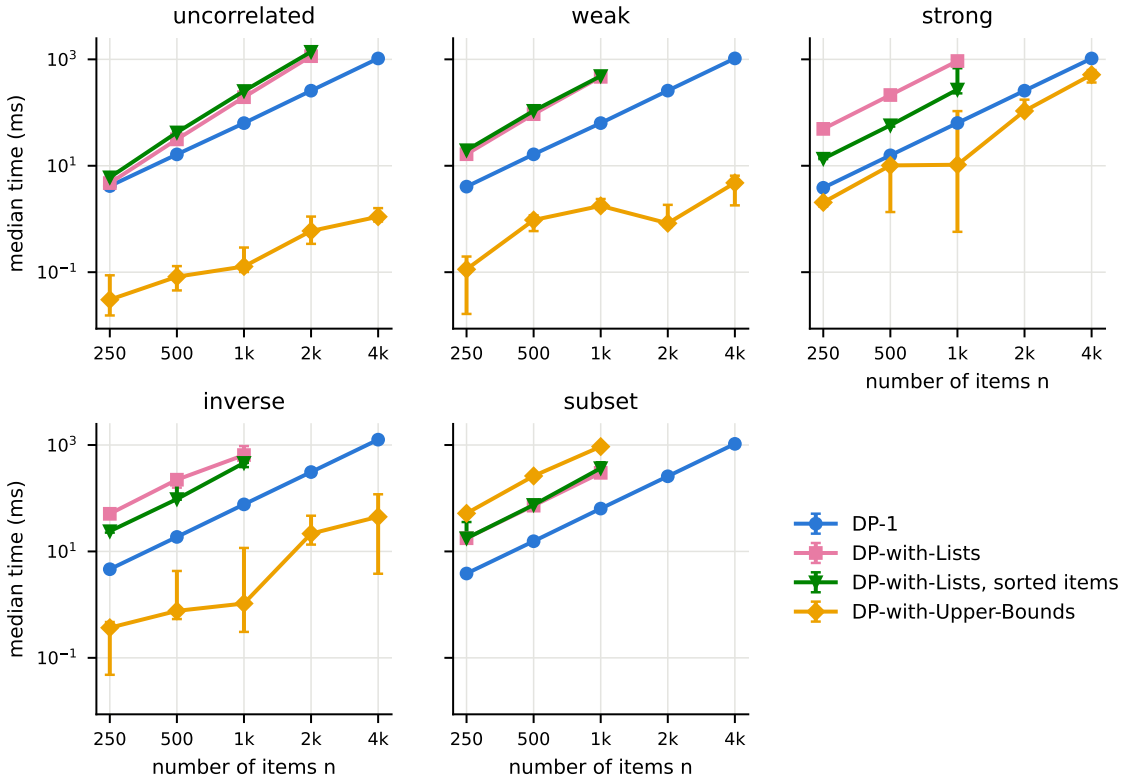


Figure 1: Experiment 1: median time against the number of items, R = 1000

3.6 Experiment 2: Weight Range

We fix $n = 1000$ and use $R = 10^2, 10^3, \ldots, 10^6$, so the capacity grows from about 2.6×10^4 to 2.5×10^8 while the number of items stays the same. The table shows the median time in ms for the uncorrelated type, together with the total number of states of DP-with-Lists.

R	Capacity c	DP-1	Lists	Bounds	Lists states
10^2	2.6×10^4	6.66	52.8	0.050	1.3×10^7
10^3	2.5×10^5	64.9	196	0.130	3.3×10^7
10^4	2.5×10^6	690	261	0.259	4.0×10^7
10^5	2.5×10^7	–	255	0.288	3.9×10^7
10^6	2.5×10^8	–	275	0.139	4.2×10^7

For weak data DP-with-Upper-Bounds stays near 2 ms up to $R = 10^6$. For inverse data it reaches 450 ms at $R = 10^5$, and for strong data 624 ms at $R = 10^4$; larger inputs are not run.

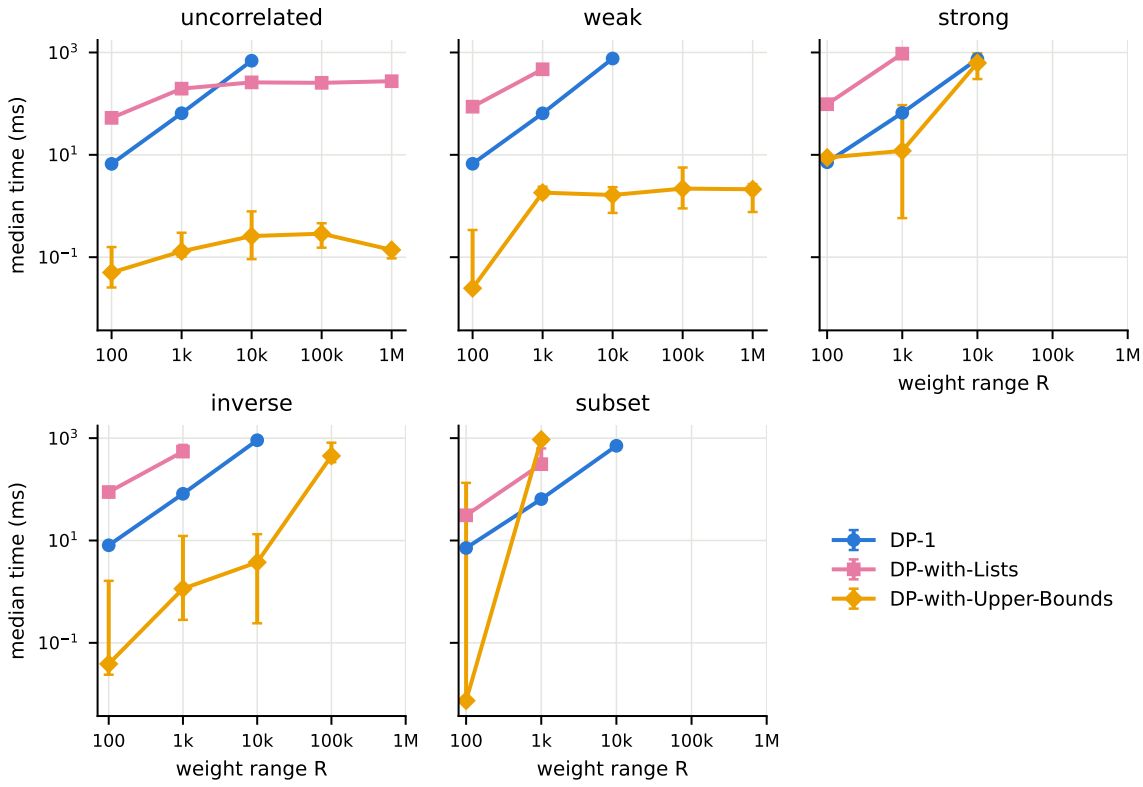


Figure 2: Experiment 2: median time against the weight range, n = 1000

3.7 Experiment 3: Many Small Items (Bonus)

For the bonus method we fix $R = 100$ (so $w \leq 100$, and $w \leq 110$ for the inverse type) and use $n = 10^3, 10^4, 10^5, 10^6$. The capacity grows from about 2.6×10^4 to 2.5×10^7. DP-1 is only run up to $n = 10^4$, where it already takes 650 to 810 ms. The table compares the two other methods (median time in ms).

Type	Bounds, $n = 10^4$	Small, $n = 10^4$	Bounds, $n = 10^6$	Small, $n = 10^6$
uncorrelated	0.381	71.2	57.2	87.9
weak	0.345	71.0	50.3	83.4
strong	115	36.4	–	49.8
inverse	0.826	44.9	1,913	57.3
subset	0.090	64.2	11.9	71.9

For the strong type, DP-with-Upper-Bounds takes 2,084 ms at $n = 10^5$ and is not run at $n = 10^6$.

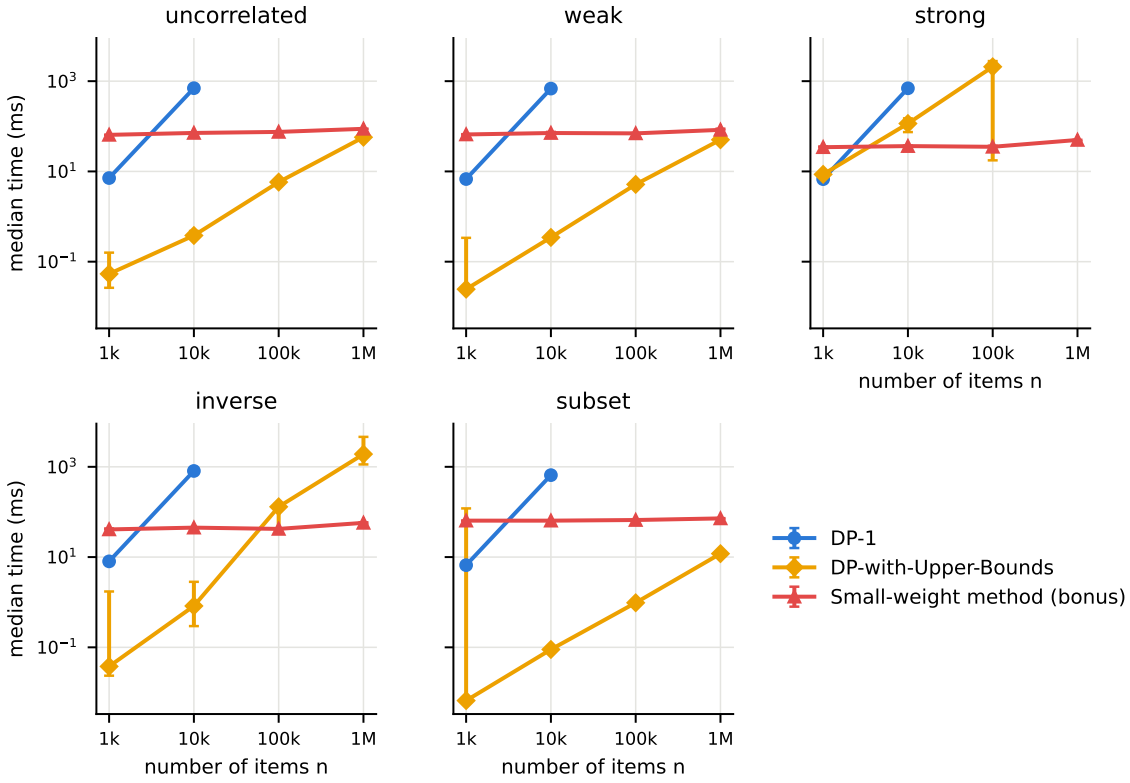


Figure 3: Experiment 3: median time against the number of items, R = 100

3.8 Experiment 4: Largest Weight (Bonus)

To see the w^3 term, we fix $n = 10^4$ and double the weight range: $R = 10, 20, 40, 80, 160, 320$. The table gives the median time in ms: DP-1 and the small-weight method for the uncorrelated type, and DP-with-Upper-Bounds for the strong type, where it is slowest. The last column is the work of the small-weight method on uncorrelated data.

R	DP-1, uncorr.	Bounds, strong	Small, uncorr.	Small, strong	Small work, uncorr.
10	71.6	1.58	0.245	0.182	1.4×10^4
20	137	0.510	1.01	0.546	4.1×10^4
40	264	96.2	6.09	2.99	2.6×10^5
80	536	165	38.8	19.7	2.0×10^6
160	–	338	256	133	1.6×10^7
320	–	207	1,810	962	1.3×10^8

On the other four types DP-with-Upper-Bounds needs at most 99 ms for every R.

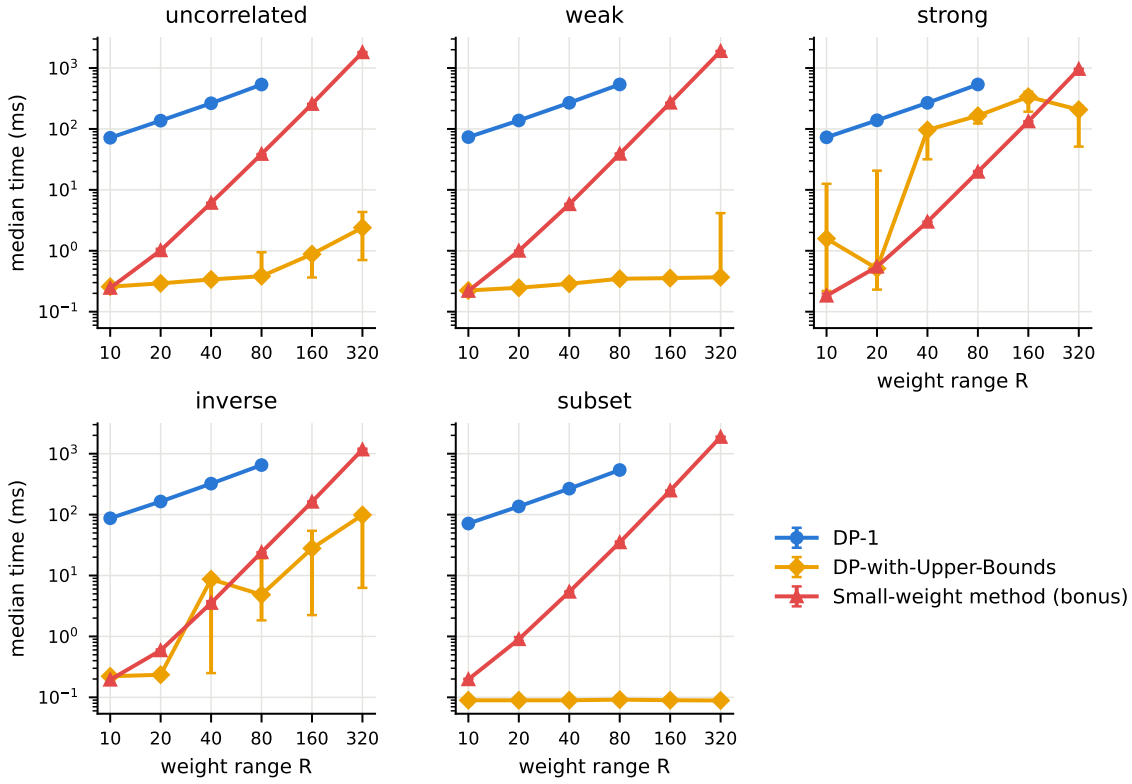


Figure 4: Experiment 4: median time against the weight range, n = 10,000

4 Analysis and Comments

4.1 Time Complexity

DP-1. Each item updates at most $c + 1$ entries, so the time is $\Theta(nc)$ in every case.

DP-with-Lists. Adding an item and merging are both linear in the list length. If B_j is the length of L_j, the time is $O(n + \sum_j B_j)$. Since $B_j \leq c + 1$, the worst case is $O(nc)$, and if every list has at most B states, the time is $O(nB)$ [1]. The states of one list have different weights and different profits, so B_j is also at most 2^j and at most the number of different profit sums. So B can be much smaller than c when c is large but the items are few. The best case is $O(n)$ [1], for example when every item has weight c.

DP-with-Upper-Bounds. Sorting takes $O(n \log n)$, and each bound needs one binary search, $O(\log n)$. If S states are created in total, the time is $O(n \log n + S \log n)$, which is $O(nc \log n)$ when nothing is removed. The book computes each bound in $O(n)$ time and gets $O(n^2 c)$ [1]; prefix sums make our bound cheaper. As in branch and bound, the worst case says little, because S is tiny when the bound is tight.

Small-weight method. The greedy prefix takes expected $O(n)$ time, and putting the items into weight classes takes $O(n + w)$. For one weight h we call SMAWK once for each remainder $r < h$, on about T/h positions, and SMAWK takes linear time [3]. So one weight costs $O(T + h)$, and all w weights cost $O(w(T + w)) = O(w^3)$ since $T < 2w^2 + w$. Sorting the chosen items takes $O(w^2 \log^2 w)$, which is smaller. The total expected time is $O(n + w^3)$, as in [2]; [2] finds the prefix with a deterministic linear-time median instead of a random pivot.

4.2 Space Complexity

DP-1 needs $O(c)$ for one row. The two list methods keep only the current and the next list, so they need $O(n + B)$ with $B \leq c + 1$. The small-weight method needs $O(n)$ for the items and $O(w^2)$ for its arrays, so $O(n + w^2)$ in total. It is the only method whose memory does not grow with c.

4.3 Discussion of Test Results

DP-1 and the lists. DP-1 does the same work on every type of data: its time grows about 4 times when n doubles (since c doubles too) and 10 times when R grows by 10, at about 0.27 ns per table entry. DP-with-Lists keeps only 13% to 74% of these entries in Experiment 1, but it is still 3 to 15 times slower, because one state costs 1.6 to 6 ns: it is written twice, takes 16 bytes, and the merge compares two lists, while the DP-1 loop is very simple. The lists only win when c is much larger than the number of useful states. In Experiment 2 the uncorrelated lists stay near 4×10^7 states and 260 ms for $R \geq 10^4$, while DP-1 grows with c and would need about 65 s at $R = 10^6$; this is the $O(nB)$ bound of Lemma 3.4.3 in [1], where B does not depend on c.

The upper bound on easy data. Sorting alone changes the number of states by at most about one third (column "Lists, sorted"), so the speed-up really comes from the bound. When the efficiencies of the items are spread out, the bound is close to the true optimum, and soon after each step only states that differ in items close to the last item that fits survive. On uncorrelated data with $n = 1000$ the states drop from 4.4×10^7 to 5.2×10^3, and on uncorrelated and weak data DP-with-Upper-Bounds is 17 to 2,700 times faster than DP-1 in Experiments 1 and 2.

The upper bound on hard data. In the subset type every item has efficiency 1, so the bound of every state is simply c, and nothing can be removed until some state fills the knapsack exactly. At $n = 1000$ this took 6.4×10^7 states and 934 ms, 15 times slower than DP-1; with many small items (Experiment 3) the greedy start usually fills the knapsack exactly, and the method stops after 3 states. Strong and inverse data are also hard to predict: for strong data with $n = 10^5$ the three inputs took 17.6 ms, 2.08 s and 2.79 s.

The small-weight method and n. In Experiment 3, n grows 1,000 times and c from 2.6×10^4 to 2.5×10^7, but the time of the small-weight method only grows from 34 to 50 ms on strong data and from 65 to 88 ms on uncorrelated data. Its work stays between 2×10^6 and 5×10^6 entries, which is set by $w = 100$. When the greedy start is already optimal, DP-with-Upper-Bounds only sorts the items and is a little faster. On strong and inverse data with $n \geq 10^5$, however, the small-weight method is up to 60 times faster, for example 57 ms against 1.9 s on inverse data with $n = 10^6$.

The small-weight method and w. In Experiment 4, each time R doubles, its work grows 7.8 to 7.9 times for $R \geq 40$, close to $2^3 = 8$, and the slope of its time on the log scale rises to 2.8. So it is 14 to 290 times faster than DP-1 for $R \leq 80$, and on strong data it is about as fast as DP-with-Upper-Bounds or faster up to $R = 160$, but at $R = 320$ it is 4.6 times slower. It is the best choice when the weights are small and the data is hard for the bound, which is the case that [2] was designed for.

4.4 Limitations and Improvements

1. The bound methods depend on a good first solution z, and our greedy start is weak on strong and subset data. Filling the room left by the greedy solution with a small DP over the items near the last one that fits would help these cases.
2. The small-weight method always uses arrays of length about $2w^2$ and one convolution per weight, so it is slow for w above a few hundred even on easy inputs. Adding weights with very few items by plain DP, or a tighter proximity bound, would cut this cost.
3. All times come from one machine and three inputs per setting, so the crossing points between the methods may move on other hardware. The work counts, however, are the same on every machine.

5 Conclusion

We implemented DP-1, the two list methods of [1] and the small-weight method of [2], and checked that they always give the same answer. DP-1 is simple, and its time depends only on n and c. Keeping only useful states pays off only when c is very large, because one state costs much more than one table entry. The upper bound gives by far the largest speed-up on uncorrelated and weakly correlated data, but it can be slower than DP-1 on subset-sum data and is hard to predict on strongly correlated data. The small-weight method avoids this problem: its time depends only on n and the largest weight, and our measurements confirm that it grows like w^3.

6 References

[1] H. Kellerer, U. Pferschy, and D. Pisinger, *Knapsack Problems*. Berlin, Heidelberg, Germany: Springer, 2004, ch. 3, sec. 3.4–3.5, pp. 50–54. doi:10.1007/978-3-540-24777-7.

[2] A. Polak, L. Rohwedder, and K. Węgrzycki, "Knapsack and Subset Sum with Small Items," in *48th International Colloquium on Automata, Languages, and Programming (ICALP 2021)*, LIPIcs, vol. 198. Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2021, pp. 106:1–106:19. doi:10.4230/LIPIcs.ICALP.2021.106.

[3] A. Aggarwal, M. M. Klawe, S. Moran, P. Shor, and R. Wilber, "Geometric applications of a matrix-searching algorithm," *Algorithmica*, vol. 2, no. 1–4, pp. 195–208, 1987. doi:10.1007/BF01840359.

[4] D. Pisinger, "Where are the hard knapsack problems?," *Computers & Operations Research*, vol. 32, no. 9, pp. 2271–2284, 2005. doi:10.1016/j.cor.2004.03.002.

7 Source Code Appendix

The source code is in the `code/` folder of the submission. `code/knapsack.cpp` contains the four algorithms: `dp1`, `dp_lists`, `dp_bounds` and `small_items`, with `merge_lists`, `smawk` and `concave_convolution` as helpers; `code/knapsack.h` declares them. `code/main.cpp` solves one instance, `code/bench.cpp` is the performance test program, and `code/gen.cpp` is the test data generator. The `README.md` shows how to compile and run them.